

Coding Agents Need a Theory of the Codebase, Not Just Code Generation

A Position Paper

1 Introduction

Coding agents built on frontier language models now resolve a majority of curated software-engineering benchmark tasks, yet practitioner reports and the benchmarks themselves increasingly point to a narrower achievement than headline numbers suggest. Most agent evaluations, including the standard SWE-bench-style suites that have driven recent progress, still treat each task as an independent episode, evaluating knowledge stored in model weights or held briefly in the input window, but not how an agent learns from and reuses experience across tasks [Zhu et al., 2026]. This is not an incidental gap in evaluation design. It reflects how these agents are built: each session begins with a fresh read of the repository’s current text, and ends with no persistent update to any model of that repository. The benchmark limitation is a symptom of a representational one.

This paper argues that the symptom has a specific, identifiable cause. Coding agents, however sophisticated their retrieval or however large their context windows, still fundamentally treat a codebase as a body of text to be read and produced. Even the more advanced systems that move beyond flat text — building dependency graphs, call graphs, or AST-derived structures — model the codebase as an *artifact*: a snapshot of what the code currently is. None of these representations capture *why* the code is that way, *which* of its structural properties are load-bearing constraints versus historical accident, or *how* the team’s conventions have shifted over the repository’s history. Call this the distinction between an artifact and the decisions that produced it. A dependency graph, however complete, is a graph of the artifact. It is categorically not a graph of the decisions, constraints, and conventions that shaped it — and it is exactly this second layer that determines whether an agent’s next change is competent or subtly wrong in a way no test suite catches.

The claim is not that structural modeling is worthless — the opposite: it is a necessary precondition, and the systems that build it represent genuine progress over agents that see only raw file text. The claim is that it is insufficient, and that its insufficiency is precisely why agents that perform well on curated, single-repository benchmarks degrade outside them. Toy repositories and heavily-curated benchmark instances have thin history, few competing conventions, and little accumulated tacit constraint — so a model of the artifact alone goes a long way. Real, long-lived codebases are the opposite: their behavior is governed as much by what a graph cannot show as by what it can.

The rest of this paper develops this distinction and defends it against the most natural objection. Section 2 surveys the structural camp and states, precisely, what its representations do and do not encode. Section 3 develops the artifact-versus-decisions distinction through three concrete failure modes — intent, constraint provenance, and convention drift — that no structural graph can represent by construction. Section 4 turns to a thinner but directly relevant literature that already gestures at this gap, usually while describing it as an engineering problem rather than a representational one. Section 5 confronts the strongest counterargument directly: that sufficiently rich retrieval over commit history, review discussion, and documentation could recover intent without requiring any new representation at all. We argue retrieval is a

similarity operation, and the specific failures that matter here — precedence, cross-episode conflict, de-feasibility — are not similarity problems, and give falsifiable criteria for telling the two apart. Section 6 sketches, at the level appropriate to a position paper, what an explicit theory of the codebase would need to represent. Section 7 addresses secondary objections. We conclude that closing this gap is not a matter of scaling current approaches, but of building agents a genuinely different kind of model — one of decisions, not just artifacts.

2 The Structure Camp: Necessary but Insufficient

The most active line of work on repository-level coding agents treats the core problem as one of representation-for-retrieval: a codebase is too large to fit in context, so the question becomes how to structure it so the right pieces surface at the right time. The dominant answer has been graphs built over the code’s static structure — call graphs, import graphs, class hierarchies, AST-derived dependency edges.

RepoGraph and CodexGraph established the basic pattern: parse the repository into a graph of entities (files, classes, functions) and relations (imports, inheritance, invocation), then let retrieval or agent traversal operate over that graph instead of flat file search. RepoGraph performs subgraph retrieval, extracting an ego-network of relevant lines and their neighbors to provide structured context, while CodexGraph indexes the repository into a graph database and has agents query it directly with a structured query language [Ouyang et al., 2025, Liu et al., 2024] — trading retrieval precision for dependence on the agent’s own querying ability. LocAgent extends this toward localization specifically, and notes a real limitation in prior work worth carrying forward here: methods like RepoGraph and CodexGraph focus on providing relevant context but do not enhance the traversal process itself, since they do not explicitly model directory structure or file hierarchies [LocAgent Authors, 2025] — a limitation entirely internal to the structure camp’s own goals, not yet the objection this paper is building toward.

The strongest result in this camp is Code Graph Models (CGM), which integrates graph structure directly into the model’s attention mechanism rather than treating it as an external retrieval index. CGM enables LLMs to comprehend functions and files through both semantic information and structural dependencies, mapping graph node attributes into the model’s input space via a specialized adapter, and combined with an agentless graph-RAG framework achieves a 43% resolution rate on SWE-bench Lite using an open-source 72B model [CGM Authors, 2025] — without any agent loop at all. This is worth taking seriously as a steelman: a substantial share of repository-level task performance is recoverable purely from structural and semantic graph information, with no explicit modeling of intent, history, or convention whatsoever. GraphCodeAgent and similar dual-graph approaches push further in the same direction, layering additional graph types (data-flow, coarse-to-fine retrieval hierarchies) over the same basic commitment: the repository is representable as a graph over its current code [Li et al., 2025].

What all of these systems encode, however precisely, is a single layer of information: the structure of the artifact as it exists right now. An import edge tells an agent that module A depends on module B. It does not tell the agent why that dependency was introduced, whether it was a deliberate architectural boundary or an expedient shortcut someone intended to remove, or whether a prior attempt to remove it was reverted for a reason no longer visible anywhere in the current tree. A call graph shows that a wrapper function sits between a client and an underlying API. It cannot show that the wrapper exists to satisfy a compliance requirement added after an incident, that removing it would silently reintroduce a resolved bug, or that three prior PRs attempting to simplify it were rejected in review for reasons never encoded into the code itself. These are not gaps in any particular graph construction — RepoGraph, CodexGraph, and CGM could all be extended with more edge types, more node attributes, richer semantic embeddings, and none of it would touch this layer, because it is not a structural property of the artifact at all. It is a property of the history that produced the artifact and the constraints that will govern its future.

This is the claim the rest of the paper turns on: structural graphs, however complete, are graphs of what the code is. They are not graphs of the decisions, constraints, and conventions that made it that way — and that second layer is not a finer-grained version of the first. It is a different kind of object, built from a different kind of evidence (commit rationale, review discussion, incident history, team convention over time), and no amount of refinement to the artifact-graph closes the distance to it. Section 3 develops this distinction through three concrete cases where the absence is not a matter of degree but of kind.

3 What Structural Graphs Cannot Represent

Section 2 argued that structural graphs encode the artifact, not the decisions behind it. This section makes that distinction concrete through three cases where the missing information is not a finer-grained structural fact but a different kind of fact altogether — one no amount of graph refinement recovers, because the graph was never built from the evidence that contains it.

3.1 Intent

A call graph records that function `validate_input` sits between the API boundary and the business logic it protects. It records this with perfect fidelity — the edge exists, the invocation is real, the structure is not in dispute. What the graph cannot record is why the function exists in that position rather than, say, inlined into the caller or pushed down into the data layer. Perhaps it was added after a specific class of malformed request caused a production incident, and its current placement is a direct response to where in the pipeline that incident occurred. Perhaps it was added speculatively by a departed engineer and never actually exercised by a real attack path. Both possibilities produce identical graph structure. An agent asked to refactor the boundary layer has no structural signal distinguishing “this placement is load-bearing” from “this placement is incidental,” because intent was never a property of the artifact — it was a property of a decision made at a point in time, and the artifact is only that decision’s residue. The information that would resolve the ambiguity — an incident report, a commit message, a review comment explaining the placement — exists, typically, but not in any form a dependency graph indexes.

3.2 Constraint provenance

A related but distinct failure: even when an agent correctly identifies that some structural property is deliberate, structure alone cannot tell it which deliberate properties are still binding. Codebases accumulate invariants whose original justification has since become obsolete, alongside invariants whose justification is as live as the day they were introduced, and the two are structurally indistinguishable — both simply appear as some property the current code satisfies. A hard-coded retry limit might be a deliberate safeguard against a rate limit that no longer exists, or it might be protecting against a vendor quirk that is still very much present. A graph edge can tell an agent that the limit exists and where it’s used. Only provenance — the commit that introduced it, the discussion that set the number, whether that discussion is still relevant to the current dependency version — can tell an agent whether it is safe to change. This is the layer institutional-knowledge framings gesture at when they describe agents repeatedly violating organization-specific conventions that require a developer to correct the agent, only for the agent to encounter another unknown constraint on the next iteration (Section 4) — each correction is evidence of exactly this gap between what is structurally present and what is actually still required.

3.3 Convention drift

The first two cases concern facts about a single point in the codebase’s history. Convention drift concerns facts about how that history has moved. Team conventions — naming schemes, error-handling patterns, module boundaries, testing expectations — are not fixed properties of a codebase; they evolve, sometimes gradually and sometimes in a deliberate migration that leaves old and new patterns coexisting for months. A structural graph built from the current tree has no way to represent this at all, because it has no temporal dimension by construction: it is a snapshot. Worse, a snapshot of a codebase mid-migration presents both the old and new convention as equally structurally valid, with nothing in the graph indicating which one is current policy and which is legacy debt awaiting cleanup. An agent generating new code has no structural basis for choosing between two patterns it can both see used elsewhere in the repository, even though a human contributor with two months of tenure would know without being told which one to follow.

3.4 The common thread

These three cases are not three degrees of the same missing structural detail — they are evidence that the object needed here is not a more detailed artifact-graph at all. Intent, provenance, and convention state are properties of a process extended in time — decisions made, revised, and superseded — not properties of the artifact that process currently happens to have produced. This is the sense in which the artifact-versus-decisions distinction is categorical rather than a matter of resolution: no edge type, no node attribute, and no amount of semantic embedding added to a dependency graph converts a snapshot into a history, because a snapshot, by definition, has already discarded the information a history would need to encode. Section 4 turns to the thinner literature that has already begun treating this as a distinct problem, generally while framing it as a memory or context-engineering fix rather than naming the representational gap directly.

4 The Intent/History/Convention Camp: Thin but Pointed the Right Way

A smaller and more scattered body of work has been converging on the same underlying move: raw historical text — commits, corrections, convention files — turns out not to be usable as-is, and each of the four lines below is a different response to discovering that. Two build a distillation step; one measures the cost of skipping it; one proposes a benchmark for whether the missing information can be recovered at all. None frames itself this way. That is the pattern this section is naming.

4.1 Distillation over raw history

MemCoder is the clearest case. Rather than treating a repository’s commit history as more text to retrieve, it argues explicitly that natural language instructions alone are insufficient to convey tacit knowledge such as complex inter-file dependencies and unwritten project conventions that developer experience encodes, and that raw human experiences are permeated with non-standardized, ambiguous, and idiosyncratic descriptions that introduce bias during an agent’s comprehension, requiring an LLM-driven construction algorithm to transform commits into standardized, agent-centric memory representations [MemCoder Authors, 2026]. The system’s premise is not that commit history is under-retrieved — it is that commit history, retrieved perfectly, is still the wrong shape to use directly. CommitDistill makes the same architectural bet from a different angle, adapting the position that agent memory should be distilled, typed knowledge rather than raw interaction text [Chukkapalli et al., 2026] specifically to a repository’s own git history, under a deliberately constrained, dependency-free execution model. Neither paper frames itself as building “a theory of the codebase”; both frame the problem as memory engineering — but the underlying commitment is the one

Section 3 argued for: history has to be converted into a structured, queryable object before it does useful work, because it is not usable in raw form.

4.2 Naming the cost of not having one

Where MemCoder and CommitDistill respond to raw-history insufficiency by building a distillation layer, this strand names what happens when no such layer exists. The “institutional knowledge tax” framing describes a recurring failure pattern: an agent’s initial attempt violates an organization-specific convention, the user corrects the error, the agent adjusts but encounters another unknown constraint, and a further correction follows [Knowledge Activation Authors, 2026] — with each cycle consuming context on failed attempts rather than progress. The paper grounds this in an existing theory of organizational learning, arguing that an organization’s ability to assimilate new knowledge depends on prior related knowledge, so a newcomer lacking organizational context faces a structural deficit no amount of technical skill can compensate for [Cohen and Levinthal, 1990] — and observes that the engineers best positioned to correct these gaps are the same senior engineers whose tacit expertise is being taxed by every correction. This is useful to the paper’s argument in a specific way: it is evidence of Section 3’s provenance and convention-drift failures showing up as a measured productivity cost in practice, independent of any particular graph-based or memory-based fix.

4.3 The field’s own stopgap

A simpler response to the same insufficiency, adopted widely without the machinery of either distillation or diagnosis, is to hand the agent prose directly: convention files like AGENTS.md, which teams write by hand to tell an agent what a graph can’t show it. An empirical evaluation asks directly whether repository-level context files are actually helpful for coding agents [AGENTS.md Evaluation Authors, 2026] — worth citing here not for its specific findings but for what its existence indicates: practitioners have already identified that something needs to sit alongside the code carrying exactly the kind of information Section 3 describes, and the field’s current answer is a static, manually maintained text file rather than any queryable, updatable representation. This is a stopgap in the precise sense that it addresses the symptom (agents lack convention knowledge) without addressing the representational problem (that knowledge has no persistence, versioning, or update mechanism — it goes stale the moment convention drifts, with nothing to signal that it has).

4.4 A convergent framing from outside the graph literature

The same insufficiency shows up independently in benchmark design, arrived at from provenance data rather than architecture. Describing what it calls the agentic development lifecycle, one paper observes that git still runs the code’s lifecycle, but a growing share of the intent behind each change is produced in agent sessions git never sees [RekalBench Authors, 2026] — and proposes a benchmark built directly from session-to-commit provenance data to measure whether agents can recover that intent after the fact. This is close to a restatement of this paper’s thesis, arrived at independently, from benchmark design rather than architecture: the concern is not that intent is hard to retrieve, but that it is actively being generated and then discarded, because nothing in the current pipeline is designed to keep it.

4.5 Why this camp stays thin

None of these four lines of work cite each other as a coherent program, and none frames its contribution as “the codebase needs a theory of its decisions, not just its structure.” Each treats the problem as local — a memory architecture, a cost to name, a stopgap file format, a benchmark gap — rather than as a shared representational failure with a common cause. That fragmentation is itself informative: it suggests the field

has repeatedly run into this wall from different directions without yet naming what’s on the other side of it, which is precisely the gap this paper argues needs to be named and, in Section 6, sketched directly. It is also, honestly, a citation-density risk for this paper — a thinner literature to draw on than Section 2’s — but the thinness is the finding, not an obstacle to it.

5 Why This Isn’t Just “More Retrieval”

5.1 The counterargument at full strength

A critic could argue that intent, constraint, and convention are already recoverable — not through a new representation, but through sufficiently rich retrieval over the artifacts that already encode them: commit messages, PR discussions, docstrings, incident postmortems. On this view, Section 3’s “gap” is really a retrieval-engineering gap: index more, embed better, retrieve deeper into history, and intent emerges as a byproduct. No new theory of the codebase is needed — just a better retriever.

5.2 The field’s own systems undercut this, but not in the obvious way

The interesting evidence here isn’t a benchmark showing retrieval failing outright — it’s that the systems built to solve this problem keep discovering, empirically, that raw retrieval isn’t enough and quietly build a representation layer anyway. MemCoder is the clearest case: raw human experiences are permeated with non-standardized, ambiguous, idiosyncratic descriptions that introduce bias during comprehension, so the system requires an LLM-driven construction algorithm to transform raw commits into standardized, agent-centric memory representations before they’re usable [MemCoder Authors, 2026]. That’s not a retrieval upgrade — retrieval quality was never the bottleneck being fixed. It’s the insertion of a distillation step whose entire job is to convert raw historical text into something with typed structure: a decision, a rationale, a scope. CommitDistill does the same move from the opposite direction, explicitly adapting a typed-memory argument — that agent memory should be distilled, typed knowledge rather than raw interaction text — to a repository’s own git history [Chukkapalli et al., 2026].

The pattern worth naming: practitioners keep re-discovering the representation requirement while describing it as an engineering necessity, not a conceptual one. That’s stronger evidence for this paper’s thesis than a clean ablation would be, because it shows the requirement surfacing independently, under no theoretical pressure to confirm it.

5.3 What retrieval structurally cannot do, and how to test for it

Retrieval is a similarity operation. Three failure modes require something else, and each gives a concrete, checkable prediction:

- **Precedence** — a decision was later reversed. Retrieval surfaces both the original and the reversing rationale with no encoded ordering; an agent has no way to know which is authoritative without an explicit “supersedes” relation.
- **Cross-episode conflict** — an edit touches an invariant established in a commit that is semantically unrelated to the current task’s language. Similarity search never surfaces it because nothing about the surface text overlaps; only an explicit dependency/constraint link would.
- **Defeasibility** — a general convention has a documented exception. Embedding similarity has no mechanism for “this rule, except in this case” — that’s a logical relation, not a proximity relation.

These aren't scale problems. More retrieval depth doesn't fix an ordering, conflict, or exception problem, because retrieval was never built to represent order, conflict, or exception — those require persistence and update semantics, which is a different kind of object than an index.

5.4 Honesty about the evidence

No existing benchmark directly isolates retrieval-scaling-plateau on intent-recovery tasks specifically; SWE-ContextBench's observation that agents are evaluated on knowledge in weights or short-term context but not on learning from and reusing past experience across tasks [Zhu et al., 2026] is suggestive, not dispositive. This is flagged here as a testable prediction the field hasn't yet measured directly — it hands future work (including benchmarks like the repo-grounded intent-recall one in Section 4) a clean target, rather than overclaiming a result that doesn't yet exist.

5.5 A framing device

This is structurally the RAG-vs-knowledge-base debate transplanted into code: embedding retrieval approximates a KB for similarity-shaped queries and breaks on consistency and defeasibility — the exact things KBs exist to encode. This gives a familiar handle for what “representation” means here, beyond “graph” or “retrieval.”

6 Toward a Theory of the Codebase

The preceding sections argue for a gap rather than a design. This section sketches, at the level appropriate to a position paper, what would need to be true of a representation that actually closed it — not a system architecture, but a specification of the properties any adequate representation must have, against which existing and future systems can be measured.

6.1 What the representation must capture

Section 3 identified three failure modes; each implies a minimum representational requirement, not a feature wishlist:

- **Intent** requires that decisions be first-class objects distinct from the artifact they produced — a decision has to be something the representation can point to, query, and attach a rationale to, independent of the code that resulted from it. A comment or commit message is evidence for a decision; it is not the decision represented as a queryable object.
- **Constraint provenance** requires that constraints carry a validity condition, not just an origin. It is not enough to record that a retry limit was set for a reason in 2023; the representation must support asking whether that reason still holds against the codebase's current dependencies, which means constraints need to be linked to the conditions that would invalidate them, not just to the commit that introduced them.
- **Convention drift** requires that conventions be versioned, with explicit precedence between an old and new pattern rather than both appearing as equally valid structural facts. This is the same precedence problem Section 5 identified as a failure mode retrieval cannot solve: an adequate representation needs an explicit “supersedes” relation, not just temporal metadata a retriever might or might not weight correctly.

6.2 What distinguishes this from existing memory and graph systems

The systems surveyed in Sections 2 and 4 already build parts of this. Structural graphs supply the artifact layer this representation would sit alongside, not replace — decision and constraint objects need to reference specific graph nodes to be useful. MemCoder and CommitDistill already perform the distillation step Section 4 described, converting raw history into structured objects; what they don’t yet do, based on the accounts given in Section 4, is make precedence and defeasibility first-class relations rather than implicit consequences of how memories are retrieved and ranked. The distinction this paper is proposing is narrow but load-bearing: distillation produces structured facts; a theory of the codebase additionally needs relations between those facts — supersession, conditional validity, exception — that determine which facts are currently authoritative. Section 4’s systems are necessary components; none of the accounts reviewed claims to supply this relational layer, and that absence is the specific gap this section is naming.

6.3 A test, not a design

The four properties below are not proposed as a specification — this paper takes no position on storage layer, schema, or agent architecture. They are proposed as the minimum a reviewer or practitioner should check for before accepting that a system has closed the gap described in Sections 2 through 5, as opposed to having built a better artifact graph under a new name.

- **Persistence** across sessions and tasks, so that a decision recorded in one agent session is available, unaltered, to a different session weeks later — the absence of this is what SWE-ContextBench’s observation about agents not learning from and reusing past experience across tasks is symptomatic of [Zhu et al., 2026].
- **Explicit update semantics**, so that when a decision is revised, the representation records the revision as a relation to the prior decision rather than simply overwriting it — this is what makes precedence queries answerable at all.
- **Queryability against the artifact graph**, so that structural elements (a function, a module boundary) can be queried for the decisions and constraints currently attached to them, rather than decisions living in a separate, unlinked store that an agent has to remember to consult.
- **A criticality signal**, distinguishing constraints an agent should treat as hard invariants from those it can reasonably revisit — without this, an agent has no way to calibrate how much evidence it needs before touching something, and defaults either to over-caution (refusing sensible changes) or under-caution (the brittleness this paper opened with).

6.4 What this section is not claiming

This is not a proposal for a specific data model, storage layer, or agent architecture — those are systems-paper questions, and premature commitment to one risks the same fate as AGENTS.md: a plausible-looking stopgap that solves an instance of the problem without addressing the representational requirement underneath it. The claim here is narrower and, we think, more durable: any system that wants to close the gap described in Sections 2 through 5 will need to satisfy the four properties above in some form, and evaluating a proposed system against them — rather than against benchmark resolution rate alone — is a more direct test of whether it has actually built a theory of the codebase or merely a better artifact graph.

7 Implications and Objections

7.1 “This is expensive to build and maintain”

The most immediate objection is cost: a representation with persistence, versioning, and explicit precedence relations is a maintenance burden a dependency graph is not, since graphs can be regenerated automatically from source on every commit, while decision and constraint objects require judgment calls about what’s worth recording and what supersedes what. This objection has real force and this paper doesn’t have a knockdown reply to it — only a reframing. The cost already exists; it is currently paid as the institutional knowledge tax described in Section 4, absorbed piecemeal by senior engineers correcting agents in real time, every session, indefinitely. The question is not whether this cost can be eliminated, but whether it is cheaper paid once, structurally, or paid repeatedly, as a recurring correction loop with no accumulation. A representation that fails to make maintenance cheaper than the status quo correction tax would not be worth building — that is a legitimate empirical bar for any proposed system, not a reason to abandon the goal.

7.2 “It will go stale, and staleness is worse than absence”

A closely related worry: an explicit model of constraints and conventions might be more dangerous than no model at all, because an agent that trusts a stale “this is still load-bearing” annotation will act with false confidence, whereas an agent with no such annotation is at least forced toward caution or a query to a human. This is a real risk, and it is precisely why Section 6’s criticality signal was framed as a requirement rather than a nice-to-have — the representation needs a mechanism for a constraint to expire or be flagged as unverified, not just a mechanism for recording it once. AGENTS.md-style files fail exactly this way in practice: they are written once, rarely revisited, and nothing in their format distinguishes “still true” from “true when written.” Any successor representation has to treat staleness as a first-class state, not an unfortunate side effect to be minimized later.

7.3 “Who curates this, and does that just relocate the tacit-knowledge problem?”

A more structural objection: if decisions and constraints must be recorded by someone, the representation doesn’t eliminate the need for tacit knowledge to be externalized — it just relocates the externalization from “correcting an agent live” to “authoring a decision record,” and the people with the relevant tacit knowledge are the same scarce senior engineers either way. This is fair, and the honest answer is that this paper’s claim was never that a codebase representation eliminates the need for human judgment about intent and constraint — only that it changes the economics of that judgment from a per-session, per-agent, per-correction cost to a one-time, cumulative, queryable record. Whether that shift is large enough to matter in practice is an empirical question this paper cannot settle, but it is at minimum a different bet than the status quo, which currently has no mechanism for judgment, once given, to persist at all.

7.4 What’s actually at stake

These objections share a pattern: none of them dispute that the representational gap in Section 3 is real; they dispute whether closing it is worth its cost, or whether it can be closed safely. That is the right level for the debate to be at. The alternative — treating brittleness outside benchmark repositories as a problem that better retrieval, bigger context windows, or more refined structural graphs will eventually solve — is the position this paper has argued against directly: those approaches improve the artifact-graph layer, and the artifact-graph layer was never where this problem lived.

8 Conclusion

This paper began with an anomaly — coding agents that resolve curated benchmark tasks at a high rate while degrading sharply in real, long-lived codebases — and argued that the anomaly has a specific cause rather than a scale problem. Structural graphs, however refined, represent the artifact a codebase currently is; they cannot represent the decisions, constraints, and conventions that made it that way, because that information was never structural to begin with. It lives in commit rationale, review discussion, incident history, and team convention as it shifts over time — evidence a dependency graph was never built to index, no matter how many edge types or semantic embeddings are added to it.

The literature already shows signs of running into this wall from several directions at once, usually without naming it directly: memory systems that distill raw commit history because retrieving it directly proved insufficient, a named cost for the correction cycles agents impose on senior engineers, a manually maintained convention file adopted as a stopgap, and a benchmark built specifically to measure whether agents can recover intent that git itself never captured. None of these treats itself as part of a shared program. This paper’s contribution is to name the pattern connecting them: what’s missing is not more retrieval or richer structure, but a representation of decisions as objects distinct from the artifact they produced — with persistence across sessions, explicit precedence between superseded and current conventions, and a criticality signal distinguishing load-bearing constraints from historical accident. Section 5 gave this claim a falsifiable edge against its most natural rival: retrieval is a similarity operation, and precedence, cross-episode conflict, and defeasibility are not similarity problems, whatever the size of the index behind them.

None of this is a claim that the fix is easy, cheap, or without its own risks — Section 7 took the cost and staleness objections seriously because they are serious. It is a claim about where the difficulty actually sits. Treating agent brittleness outside toy repositories as a symptom that will resolve with bigger context windows or better-structured retrieval is a bet on the wrong layer. The layer that needs modeling is not the code. It is the history of decisions that produced it, and the theory of the codebase this paper is arguing for is, at bottom, a theory of that history — one coding agents do not yet have, and current approaches, however sophisticated, are not building.

References

- AGENTS.md Evaluation Authors. Evaluating agents.md: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*, 2026.
- CGM Authors. Code graph model (cgm): A graph-integrated large language model for repository-level software engineering tasks. In *NeurIPS 2025*, 2025. OpenReview: b98ODdeYq5.
- Divya Chukkapalli, Thejesh Avula, Aditya Aggarwal, Harsimran Singh, and Amith Tallanki. Commitdistill: A lightweight knowledge-centric memory layer for software repositories. *arXiv preprint arXiv:2605.18284*, 2026.
- Wesley M. Cohen and Daniel A. Levinthal. Absorptive capacity: A new perspective on learning and innovation. *Administrative Science Quarterly*, 35(1):128–152, 1990.
- Knowledge Activation Authors. Knowledge activation: Ai skills as the institutional knowledge primitive for agentic software development. *arXiv preprint arXiv:2603.14805*, 2026.
- Jia Li, Xianjie Shi, Kechi Zhang, Ge Li, Zhi Jin, Lei Li, Huangzhao Zhang, Jia Li, Fang Liu, Yuwei Zhang, Zhengwei Tao, Yihong Dong, Yuqi Zhu, and Chongyang Tao. Graphcodeagent: Dual graph-guided llm agent for retrieval-augmented repo-level code generation. *arXiv preprint arXiv:2504.10046*, 2025.

Xiangyan Liu et al. Codexgraph: Bridging large language models and code repositories via code graph databases, 2024. *arXiv preprint*.

LocAgent Authors. Locagent: Graph-guided llm agents for code localization. *arXiv preprint arXiv:2503.09089*, 2025.

MemCoder Authors. Your code agent can grow alongside you with structured memory. *arXiv preprint arXiv:2603.13258*, 2026.

Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing ai software engineering with repository-level code graph, 2025. *arXiv:2410.14709*.

RekalBench Authors. The question an agent actually asks. *arXiv preprint arXiv:2607.14390*, 2026.

Jared Zhu, Minhao Hu, and Junde Wu. Swe context bench: A benchmark for context learning in coding. *arXiv preprint arXiv:2602.08316*, 2026.